

Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations

Philippe Tillet
Harvard University
USA

H. T. Kung
Harvard University
USA

David Cox
Harvard University, IBM
USA

Abstract

The validation and deployment of novel research ideas in the field of Deep Learning is often limited by the availability of efficient compute kernels for certain basic primitives. In particular, operations that cannot leverage existing vendor libraries (e.g., cuBLAS, cuDNN) are at risk of facing poor device utilization unless custom implementations are written by experts – usually at the expense of portability. For this reason, the development of new programming abstractions for specifying custom Deep Learning workloads at a minimal performance cost has become crucial.

We present *Triton*, a language and compiler centered around the concept of *tile*, i.e., statically shaped multi-dimensional sub-arrays. Our approach revolves around (1) a C-based language and an LLVM-based intermediate representation (IR) for expressing tensor programs in terms of operations on parametric tile variables and (2) a set of novel tile-level optimization passes for compiling these programs into efficient GPU code. We demonstrate how Triton can be used to build portable implementations of matrix multiplication and convolution kernels on par with hand-tuned vendor libraries (cuBLAS / cuDNN), or for efficiently implementing recent research ideas such as shift convolutions.

CCS Concepts • **Computing methodologies** → **Parallel computing methodologies**.

Keywords compiler; neural networks; GPU

ACM Reference Format:

Philippe Tillet, H. T. Kung, and David Cox. 2019. Triton: An Intermediate Language and Compiler for Tiled Neural Network Computations. In *Proceedings of the 3rd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL '19)*, June 22, 2019, Phoenix, AZ, USA. ACM, New York, NY, USA, 10 pages. <https://doi.org/10.1145/3315508.3329973>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. MAPL '19, June 22, 2019, Phoenix, AZ, USA

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-6719-6/19/06...\$15.00
<https://doi.org/10.1145/3315508.3329973>

1 Introduction

The recent resurgence of Deep Neural Networks (DNNs) was largely enabled [24] by the widespread availability of programmable, parallel computing devices. In particular, continuous improvements in the performance of many-core architectures (e.g., GPUs) have played a fundamental role, by enabling researchers and engineers to explore an ever-growing variety of increasingly large models, using more and more data. This effort was supported by a collection of vendor libraries (cuBLAS, cuDNN) aimed at bringing the latest hardware innovations to practitioners as quickly as possible. Unfortunately, these libraries only support a restricted set of tensor operations, leaving the implementation of novel primitives to experts [13, 17, 25].

This observation has led to the development of various Domain-Specific Languages (DSLs) for DNNs, based on polyhedral machinery (e.g., Tensor Comprehensions [43]) and/or loop synthesis techniques (e.g., Halide [37], TVM [10] and PlaidML [22]). But while these systems generally perform well for certain classes of problems such as depthwise-separable convolutions (e.g., MobileNet [20]), they are often much slower than vendor libraries in practice (see, e.g., Figure 1), and lack the expressivity necessary to implement structured sparsity patterns [28, 31, 47] that cannot be directly specified using affine array indices in nested loops.

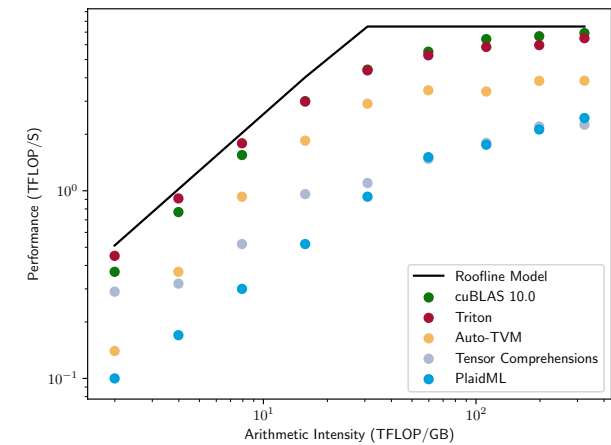


Figure 1. Performance of various implementations of $C = AB^T$ vs. Roofline [46] Model (NVIDIA GeForce GTX1070), where $A \in \mathbb{R}^{1760 \times 1760}$ and $B \in \mathbb{R}^{N \times 1760}$ as N modulates arithmetic intensity.

Triton: 一种用于分块神经网络计算的中间语言与编译器

菲利普·蒂耶
哈佛大学 美国

H. T. Kung
美国哈佛大学

大卫·考克斯
哈佛大学, IBM 美国

摘要

深度学习领域新颖研究想法的验证与部署，常受限于特定基础原语高效计算内核的可用性。特别是那些无法利用现有厂商库（如cuBLAS、cuDNN）的运算，除非由专家编写定制化实现——通常以牺牲可移植性为代价——否则可能面临设备利用率低下的风险。因此，开发能以最小性能代价指定定制化深度学习工作负载的新型编程抽象已变得至关重要。

我们推出了Triton，这是一种以瓦片（即静态形状的多维数组）概念为核心的语言和编译器。我们的方法围绕以下两点展开：(1) 一种基于C的语言和基于LLVM的中间表示（IR），用于通过参数化瓦片变量的操作来表达张量程序；(2) 一套新颖的瓦片级优化流程，用于将这些程序编译成高效的GPU代码。我们展示了如何利用Triton构建可移植的矩阵乘法 and 卷积内核实现，其性能可与手动调优的供应商库（cuBLAS / cuDNN）相媲美，或用于高效实现诸如移位卷积等最新研究思路。

CCS概念 • 计算方法论 → 并行计算方法论。

关键词 编译器；神经网络；GPU

ACM 参考文献格式：

Philippe Tillet, H. T. Kung 和 David Cox。2019。Triton：一种用于分块神经网络计算的中间语言与编译器。载于《第三届ACM SIGPLAN机器学习与编程语言国际研讨会论文集》（MAPL '19），2019年6月22日，美国亚利桑那州凤凰城。ACM，美国纽约州纽约市，10页。 <https://doi.org/10.1145/3315508.3329973>

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than the author(s) must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org. MAPL '19, June 22, 2019, Phoenix, AZ, USA

© 2019 Copyright held by the owner/author(s). Publication rights licensed to ACM.

ACM ISBN 978-1-4503-6719-6/19/06...\$15.00
<https://doi.org/10.1145/3315508.3329973>

1 引言

深度神经网络（DNN）近期的复兴很大程度上得益于可编程并行计算设备的广泛普及[24]。特别是，众核架构（例如GPU）性能的持续提升发挥了根本性作用，使研究人员和工程师能够利用日益增多的数据，探索规模不断扩大的多样化模型。这一进程得到了供应商库（如cuBLAS、cuDNN）的支持，这些库致力于将最新的硬件创新快速交付给实践者。然而，这些库仅支持有限的张量运算，导致新型基础操作的实现仍需依赖专家完成[13, 17, 25]。

这一观察推动了多种基于多面体机制（例如Tensor Comprehensions [43]）和/或循环合成技术（例如Halide [37]、TVM [10]和PlaidML[22]）的领域特定语言（DSL）的开发。然而，尽管这些系统在特定类别的问题（如深度可分离卷积，例如MobileNet [20]）上通常表现良好，但在实践中往往比厂商库慢得多（参见图1），并且缺乏实现结构化稀疏模式[28, 31, 47]所需的表达能力，这些模式无法通过嵌套循环中的仿射数组索引直接指定。

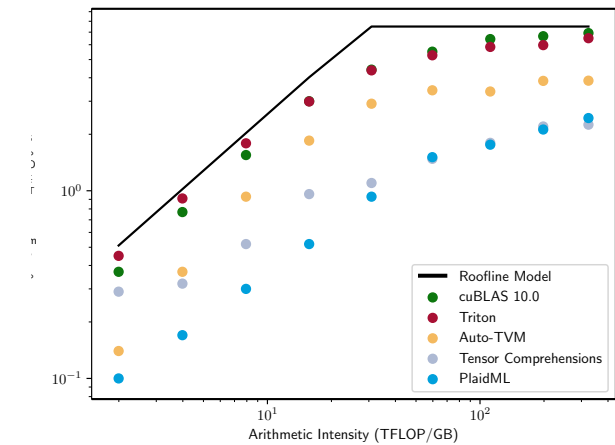


图1. $C = AB^T$ 与Roofline [46] 模型（NVIDIA GeForce GTX1070）的各种实现性能对比，其中 $A \in \mathbb{R}^{1760 \times 1760}$ 和 $B \in \mathbb{R}^{N \times 1760}$ 随 N 变化调节算术强度。

These issues have often been addressed by the use of micro-kernels [11, 21] – i.e., hand-written tile-level intrinsics – but this solution requires a lot of manual labor and lacks portability. And while several high-level programming abstractions for *tiling* have recently been proposed [23, 41], underlying compiler backends still lack support for tile-level operations and optimizations. To this end we present Triton (Figure 2), an open-source¹ intermediate language and compiler for specifying and compiling tile programs into efficient GPU code.

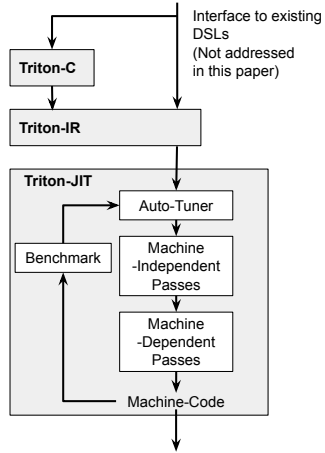


Figure 2. Overview of Triton

The main contributions of this paper are summarized as follows:

- **Triton-C** (Section 3): A C-like language for expressing tensor programs in terms of parametric tile variables. The purpose of this language is to provide a stable interface for existing DNN transcompilers (e.g., PlaidML, Tensor Comprehensions) and programmers familiar with CUDA. Listing 1 shows the Triton-C source code associated with a simple matrix multiplication task.
- **Triton-IR** (Section 4): An LLVM-based Intermediate Representation (IR) that provides an environment suitable for tile-level program analysis, transformation and optimization. Listing 5 shows the Triton-IR code for a Rectified Linear Unit (ReLU) function. Here Triton-IR programs are constructed directly from Triton-C during parsing, but automatic generation from embedded DSLs or higher-level DNN compilers (e.g., TVM) could also be explored in the future.
- **Triton-JIT** (Section 5): A Just-In-Time (JIT) compiler and code generation backend for compiling Triton-IR programs into efficient LLVM bitcode. This includes (1) a set of tile-level, machine-independent passes aimed at simplifying input compute kernels independently of

Listing 1. $C = A \times B^T$ in Triton-C. Keywords specific to Triton are shown in purple.

```

// Tile shapes are parametric and can be optimized
// by compilation backends
const tunable int TM = {16, 32, 64, 128}
const tunable int TN = {16, 32, 64, 128}
const tunable int TK = {8, 16}
// C = A * B.T
kernel void matmul_nt(float* a, float* b, float* c,
                     int M, in N, int K)
{
    // 1D tile of indices
    int rm[TM] = get_global_range(0);
    int rn[TN] = get_global_range(1);
    int rk[TK] = 0 ... TK;
    // 2D tile of accumulators
    float C[TM, TN] = 0;
    // 2D tile of pointers
    float* pa[TM, TK] = a + rm[:, newaxis] + rk * M;
    float* pb[TN, TK] = b + rn[:, newaxis] + rk * K;
    for(int k = K; k >= 0; k -= TK){
        bool check_k[TK] = rk < k;
        bool check_a[TM, TK] = (rm < M)[:, newaxis] && check_k;
        bool check_b[TN, TK] = (rn < N)[:, newaxis] && check_k;
        // load tile operands
        float A[TM, TK] = check_a ? *pa : 0;
        float B[TN, TK] = check_b ? *pb : 0;
        // accumulate
        C += dot(A, trans(B));
        // update pointers
        pa = pa + TK*M;
        pb = pb + TK*N;
    }
    // write-back accumulators
    float* pc[TM, TN] = c + rm[:, newaxis] + rn * M;
    bool check_c[TM, TN] = (rm < M)[:, newaxis] && (rn < N);
    @check_c *pc = C;
}

```

any compilation target; (2) a set of tile-level machine-dependent passes for generating efficient GPU-ready LLVM-IR; and (3) an auto-tuner that optimize any meta-parameter associated with the above passes.

- **Numerical Experiments** (Section 6): A numerical evaluation of Triton that demonstrates its ability to (1) generate matrix multiplication implementations on par with cuBLAS and up to 3× faster than alternatives DSLs on recurrent and transformer neural networks; (2) re-implement cuDNN’s IMPLICIT_GEMM algorithm for dense convolution without performance loss; and (3) create efficient implementations of novel research ideas such as shift-conv [47] modules.

This paper will be prefaced by a brief analysis of the existing related literature (Section 2) and concluded by a summary and directions of future work (Section 7).

2 Related Work

The existence of frameworks [1, 9, 36] and libraries for Deep Learning has been critical to the emergence of novel neural network architectures and algorithms. But despite advances in analytical [5, 48] and empirical [6, 30] heuristics for linear algebra compilers, these software still invariably rely on hand-optimized sub-routines (e.g., cuBLAS and cuDNN). This has led to development of various DSLs and

这些问题通常通过使用微内核[11, 21]——即手写的瓦片级内部函数——来解决，但这种方法需要大量手动工作且缺乏可移植性。尽管最近提出了几种用于分块的高级编程抽象[23, 41]，但底层编译器后端仍然缺乏对瓦片级操作和优化的支持。为此，我们推出了Triton（图2），这是一个开源¹中间语言和编译器，用于将瓦片程序规范并编译为高效的GPU代码。

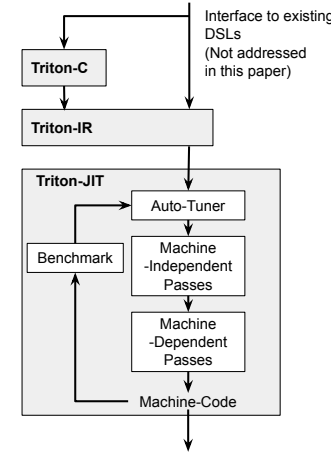


图2. Triton概览

本文的主要贡献总结如下：

- Triton-C（第3节）：一种类C语言，用于通过参数化瓦片变量表达张量程序。该语言旨在为现有的DNN转译器（如PlaidML、Tensor Comprehensions）以及熟悉CUDA的程序员提供稳定接口。清单1展示了与简单矩阵乘法任务相关的Triton-C源代码。
- Triton-IR（第4节）：一种基于LLVM的中间表示（IR），提供了一个适用于图块级程序分析、转换和优化的环境。清单5展示了一个修正线性单元（ReLU）函数的Triton-IR代码。此处Triton-IR程序在解析过程中直接从Triton-C构建，但未来也可探索从嵌入式DSL或更高级别的DNN编译器（例如TVM）自动生成。
- Triton-JIT（第5节）：一个即时（JIT）编译器和代码生成后端，用于将Triton-IR程序编译为高效的LLVM位码。这包括（1）一组瓦片级、与机器无关的传递，旨在独立于输入计算内核进行简化

清单 1. Triton-C 中的 $C = A \times B^T$ 。Triton 特有的关键字以紫色显示。

```

// Tile shapes are parametric and can be optimized
// by compilation backends
const tunable int TM = {16, 32, 64, 128}
const tunable int TN = {16, 32, 64, 128}
const tunable int TK = {8, 16}
// C = A * B.T
kernel void matmul_nt(float* a, float* b, float* c,
                     int M, in N, int K)
{
    // 1D tile of indices
    int rm[TM] = get_global_range(0);
    int rn[TN] = get_global_range(1);
    int rk[TK] = 0 ... TK;
    // 2D tile of accumulators
    float C[TM, TN] = 0;
    // 2D tile of pointers
    float* pa[TM, TK] = a + rm[:, newaxis] + rk * M;
    float* pb[TN, TK] = b + rn[:, newaxis] + rk * K;
    for(int k = K; k >= 0; k -= TK){
        bool check_k[TK] = rk < k;
        bool check_a[TM, TK] = (rm < M)[:, newaxis] && check_k;
        bool check_b[TN, TK] = (rn < N)[:, newaxis] && check_k;
        // load tile operands
        float A[TM, TK] = check_a ? *pa : 0;
        float B[TN, TK] = check_b ? *pb : 0;
        // accumulate
        C += dot(A, trans(B));
        // update pointers
        pa = pa + TK*M;
        pb = pb + TK*N;
    }
    // write-back accumulators
    float* pc[TM, TN] = c + rm[:, newaxis] + rn * M;
    bool check_c[TM, TN] = (rm < M)[:, newaxis] && (rn < N);
    @check_c *pc = C;
}

```

任何编译目标；(2) 一套面向图块的机器相关通道，用于生成高效的GPU就绪LLVM-IR；(3) 一个自动调优器，用于优化与上述通道相关的任何元参数。

- 数值实验（第6节）：对Triton进行的数值评估表明，其能够（1）生成与cuBLAS性能相当的矩阵乘法实现，在循环神经网络和Transformer神经网络上比替代DSL快达3×倍；（2）重新实现cuDNN的IMPLICIT_GEMM稠密卷积算法且无性能损失；（3）为新颖研究思路（如shift-conv[47]模块）创建高效实现。

本文将在开篇简要分析现有相关文献（第2节），并在结尾总结全文并展望未来工作方向（第7节）。

2 相关工作

深度学习框架[1, 9, 36]和库的存在对于新型神经网络架构和算法的涌现至关重要。尽管线性代数编译器的分析性[5, 48]和实证性[6, 30]启发式方法有所进步，但这些软件仍然始终依赖手动优化的子程序（例如cuBLAS和cuDNN）。这导致了各种领域特定语言（DSL）和

¹<http://triton-lang.org>

¹<http://triton-lang.org>

compilers for DNNs, generally based on one of three distinct approaches:

- **Tensor-level IRs** have been used by XLA [16] and Glow [38] to transform tensor programs into predefined LLVM-IR and CUDA-C operation templates (e.g., tensor contractions, element-wise operations, etc.) using pattern-matching.
- **The polyhedral model** [18] has been used by Tensor Comprehensions (TC) [43] and Diesel [14] to parameterize and automate the compilation of one or many DNN layers into LLVM-IR and CUDA-C programs.
- **Loop synthesizers** have been used by Halide [37] and TVM [10] to transform tensor computations into loop nests that can be manually optimized using user-defined (though possibly parametric [11]) schedules.

By contrast, Triton relies on the addition of *tile-level* operations and optimizations into traditional compilation pipelines. This approach provides (1) more flexibility than XLA and Glow; (2) support for non-affine tensor indices contrary to TC and Diesel; and (3) automatic inference of possible execution schedule that would otherwise have to be specified manually to Halide or TVM. The benefits of Triton come at the cost of increased programming efforts – see Listing 2 for implementations of matrix multiplication in these DSLs.

Listing 2. $C = A \times B^T$ in TF, PlaidML, TC and TVM

```
C = tf.matmul(A, tf.transpose(B))      // TF
C[i, j: I, J] = +(A[i, k] * B[j, k]); // PlaidML
C(i, j) +=! A(i, k) * B(j, k)          // TC
tvm.sum(A[i, k] * B[j, k], axis=k)     // TVM
```

3 The Triton-C Language

The purpose of Triton-C is to provide a stable frontend for existing (and future) DNN transcompilers, as well as programmers familiar with low-level GPU programming. In this section we describe the CUDA-like syntax of Triton-C (Section 3.1), its Numpy[35]-like semantics (Section 3.2) and its "Single-Program, Multiple-Data" (SPMD) programming model (Section 3.3).

3.1 Syntax

The syntax of Triton-C is based on that of ANSI C (and more specifically CUDA-C), but was modified and extended (see Listing 3) to accomodate the semantics and programming model described in the two next subsections. These changes fall into the following categories:

Tile declarations: We added special syntax for declaring multi-dimensional arrays (e.g., `int tile[16, 16]`) so as to emphasize their semantical difference with nested arrays found in ANSI C (e.g., `int tile[16][16]`). Tile shapes must be constant but can also be made parametric with the `tunable` keyword. One-dimensional integer tiles may be initialized using ellipses (e.g., `int range[8] = 0 ... 8`).

Listing 3. Grammar extensions for Triton-C. We assume the existence of certain C constructs shown in blue.

```
// Broadcasting semantics
slice      : ':' | 'newaxis'
slice_list : slice | slice_list ',' slice
slice_expr : postfix_expr | expr '[' slice_list ']'
// Range initialization
constant_range : expr '...' expr
// Intrinsics
global_range : 'get_global_range' '(' constant ')'
dot          : 'dot' '(' expr ',' expr ')'
trans       : 'trans' '(' expr ',' expr ')'
intrinsic_expr : global_range | dot | trans
// Predication
predicate_expr : '@' expr
// Tile extensions for abstract declarators
abstract_decl : abstract_decl | '[' constant_list ']'
// Extensions of C expressions
expr          : expr | constant_range | slice_expr
              | intrinsic_expr
// Extensions of C specifiers
storage_spec : storage_spec | 'kernel'
type_spec    : type_spec | 'tunable'
// Extensions of C statements
statement    : statement | predicate_expr statement
```

Built-in function: While common C syntax was retained for element-wise array operations (+, -, &&, *, etc.), various built-in functions (`dot`, `trans`, `get_global_range`) were added to support tile semantics (Section 3.2.1) and the SPMD programming model.

Broadcasting: N-dimensional tiles can be broadcast along any particular axis using the `newaxis` keyword and usual slicing syntax (e.g., `int broadcast [8, 8] = range[:, newaxis]` for stacking columns). Note that slicing tiles to retrieve scalars or sub-arrays is otherwise forbidden.

Predication: Basic control-flow within tile operations (Section 4.3) is achieved through the use of predicated statements via the `'@'` prefix.

3.2 Semantics

3.2.1 Tile semantics

The existence of built-in *tile* types and operations (i.e., tile semantics) in Triton-C offers two main benefits. First, it simplifies the structure of tensor programs by hiding important performance details pertaining to intra-tile memory coalescing [12], cache management [32] and specialized hardware utilization [27]. Second, it opens the door for compilers to perform these optimizations automatically, as discussed in Section 5.

3.2.2 Broadcasting Semantics

Tiles in Triton-C are strongly typed in the sense that certain instructions statically require their operands to obey strict shape constraints. For example, a scalar may not be added to an array unless it is first appropriately broadcast. *Broadcasting semantics* [35] provide a set of rules to perform these conversions implicitly (see Listing 4 for an example):

1. *Padding:* the shape of the shortest operand is left-padded with ones until both operands have the same dimensionality.

DNN编译器，通常基于三种不同方法之一：

- 张量级中间表示已被XLA [16]和Glow [38]用于通过模式匹配将张量程序转换为预定义的LLVM-IR和CUDA-C操作模板（例如张量收缩、逐元素运算等）。
- 多面体模型[18]已被Tensor Comprehensions (TC)[43]和Diesel[14]用于参数化并自动化编译一个或多个DNN层为LLVM-IR和CUDA-C程序。
- 循环合成器已被Halide[37]和TVM[10]用于将张量计算转换为循环嵌套，这些循环嵌套可以通过用户定义的（尽管可能是参数化的[11]）调度进行手动优化。

相比之下，Triton依赖于将瓦片级操作和优化添加到传统的编译流程中。这种方法提供了（1）比XLA和Glow更大的灵活性；（2）支持非仿射张量索引，这与TC和Diesel不同；以及（3）自动推断可能的执行调度，否则在Halide或TVM中需要手动指定。Triton的优势是以增加编程工作为代价的——参见清单2中这些DSL中矩阵乘法的实现。

清单2. $C = A \times B^T$ 在TF、PlaidML、TC和TVM中的实现

```
C = tf.matmul(A, tf.transpose(B))      // TF
C[i, j: I, J] = +(A[i, k] * B[j, k]); // PlaidML
C(i, j) +=! A(i, k) * B(j, k)          // TC
tvm.sum(A[i, k] * B[j, k], axis=k)     // TVM
```

3 Triton-C 语言

Triton-C 的目的是为现有（及未来）的 DNN 转译器以及熟悉低级 GPU 编程的程序员提供一个稳定的前端。在本节中，我们将描述 Triton-C 类 CUDA 的语法（第 3.1 节）、其类 Numpy[35] 的语义（第 3.2 节）以及其“单程序，多数据”（SPMD）编程模型（第 3.3 节）。

3.1 语法

Triton-C的语法基于ANSI C（更具体地说是CUDA-C），但为了适应接下来两个小节描述的语义和编程模型，我们对其进行了修改和扩展（见代码清单3）。这些改动主要涉及以下几个方面：

瓦片声明：我们为声明多维数组（例如`int tile[16, 16]`）添加了特殊语法，以强调其与ANSI C中嵌套数组（例如`int tile[16][16]`）的语义差异。瓦片形状必须是常量，但也可以通过`tunable`关键字使其参数化。一维整数瓦片可以使用省略号进行初始化（例如`int range[8] = 0 ... 8`）。

清单3. Triton-C的语法扩展。我们假设存在某些以蓝色显示的C语言结构。

```
// Broadcasting semantics
slice      : ':' | 'newaxis'
slice_list : slice | slice_list ',' slice
slice_expr : postfix_expr | expr '[' slice_list ']'
// Range initialization
constant_range : expr '...' expr
// Intrinsics
global_range : 'get_global_range' '(' constant ')'
dot          : 'dot' '(' expr ',' expr ')'
trans       : 'trans' '(' expr ',' expr ')'
intrinsic_expr : global_range | dot | trans
// Predication
predicate_expr : '@' expr
// Tile extensions for abstract declarators
abstract_decl : abstract_decl | '[' constant_list ']'
// Extensions of C expressions
expr          : expr | constant_range | slice_expr
              | intrinsic_expr
// Extensions of C specifiers
storage_spec : storage_spec | 'kernel'
type_spec    : type_spec | 'tunable'
// Extensions of C statements
statement    : statement | predicate_expr statement
```

内置函数：虽然保留了常见的C语法用于元素级数组操作（+, -, &&, *等），但添加了各种内置函数（`dot`、`trans`、`get_global_range`）以支持瓦片语义（第3.2.1节）和SPMD编程模型。

广播：N维瓦片可以使用`newaxis`关键字和常规切片语法（例如，`int broadcast [8, 8] = range[:, newaxis]`用于堆叠列）沿任何特定轴进行广播。请注意，其他情况下禁止通过切片瓦片来检索标量或子数组。

谓词化：瓦片操作内的基本控制流（第4.3节）通过使用带`@`前缀的谓词化语句实现。

3.2 语义学

3.2.1 瓦片语义

Triton-C中内置的瓦片类型和操作（即瓦片语义）的存在带来了两大主要优势。首先，它通过隐藏与瓦片内存合并[12]、缓存管理[32]以及专用硬件利用[27]相关的重要性能细节，简化了张量程序的结构。其次，它为编译器自动执行这些优化打开了大门，正如第5节所讨论的那样。

3.2.2 广播语义

Triton-C中的瓦片是强类型的，这意味着某些指令静态地要求其操作数遵守严格的形状约束。例如，标量不能直接与数组相加，除非先进行适当的广播。广播语义[35]提供了一组规则来隐式执行这些转换（示例见代码清单4）：

1. 填充：将最短操作数的形状左侧填充为一，直到两个操作数具有相同的维度。

2. *Broadcasting*: the content of both operands is replicated as many times as needed until their shape is identical; an error is emitted if this cannot be done.

Listing 4. Broadcasting semantics in practice

```
int a[16], b[32, 16], c[16, 1];
// a is first reshaped to [1, 16]
// and then broadcast to [32, 16]
int x_1[32, 16] = a[newaxis, :] + b;
// Same as above but implicitly
int x_2[32, 16] = a + b;
// a is first reshaped to [1, 16]
// a is broadcast to [16, 16]
// c is broadcast to [16, 16]
int y[16, 16] = a + c;
```

3.3 Programming Model

The execution of CUDA [33] code on GPUs is supported by an SPMD [4] programming model in which each kernel is associated with an identifiable thread-block in a so-called launch grid. The Triton programming model is similar, but each kernel is *single-threaded* – though automatically parallelized – and associated with a set of global ranges that varies from instance to instance (see Figure 3). This approach leads to simpler kernels in which CUDA-like concurrency primitives (shared memory synchronization, inter-thread communication, etc.) are inexistent.

The global ranges associated with a kernel can be queried using the `get_global_range(axis)` built-in function in order to create e.g., tiles of pointers as shown in Listing 1.

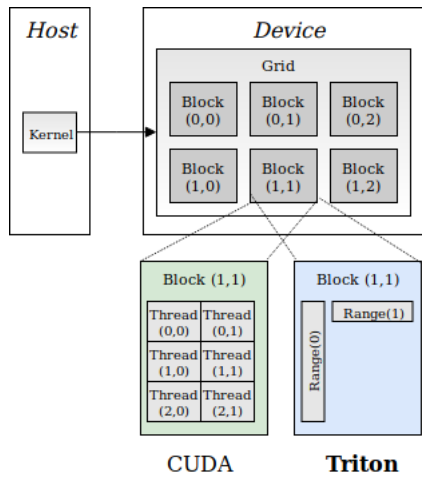


Figure 3. Difference between the CUDA and the Triton programming model

4 The Triton IR

Triton-IR is an LLVM-based Intermediate Representation (IR) whose purpose is to provide an environment suitable for tile-level program analysis, transformation and optimization. In this work, Triton-IR programs are constructed directly

Listing 5. $A = \max(A, 0)$ in Triton-IR. Note that tile shapes are non-parametric here. In this paper their values are instantiated by the Triton-JIT.

```
define kernel void @relu(float* %A, i32 %M, i32 %N) {
prologue:
  %rm = call i32<8> @get_global_range(0);
  %rn = call i32<8> @get_global_range(1);
  ; broadcast shapes
  %1 = reshape i32<8, 8> %M;
  %M0 = broadcast i32<8, 8> %1;
  %2 = reshape i32<8, 8> %N;
  %N0 = broadcast i32<8, 8> %2;
  ; broadcast global ranges
  %3 = reshape i32<8, 1> %rm;
  %rm_bc = broadcast i32<8, 8> %3;
  %4 = reshape i32<1, 8> %rn;
  %rn_bc = broadcast i32<8, 8> %4;
  ; compute mask
  %pm = icmp slt %rm_bc, %M0;
  %pn = icmp slt %rn_bc, %N0;
  %msk = and %pm, %pn;
  ; compute pointer
  %A0 = splat float*<8, 8> %A;
  %5 = getelementptr %A0, %rm_bc;
  %6 = mul %rn_bc, %M0;
  %pa = getelementptr %5, %6;
  ; compute result
  %a = load %pa;
  %_0 = splat float<8, 8> 0;
  %result = max %float %a, %_0;
  ; write back
  store fp32<8, 8> %pa, %result
}
```

from Triton-C during parsing, although they could also be generated directly from higher level DSLs in the future.

Triton-IR and LLVM-IR programs share the same high-level structure (recalled in Section 4.1), but the former also includes a number of extensions necessary for tile-level data-flow (Section 4.2) and control-flow (Section 4.3) analysis. These novel extensions are crucial for carrying out the optimizations outlined in Section 5, and for safely accessing tensors of arbitrary shapes as shown in Section 6.

4.1 Structure

4.1.1 Modules

At the highest level, Triton-IR programs consist of one or multiple basic units of compilation known as *modules*. These modules are compiled independently from one another, and eventually aggregated by a linker whose role is to resolve forward declarations and adequately merge global definitions.

Each module itself is composed of functions, global variables, constants and other miscellaneous symbols (e.g., meta-data, function attributes).

4.1.2 Functions

Triton-IR function definitions consist of a return type, a name and a potentially empty arguments list. Additional visibility, alignment and linkage specifiers can be added if desired. Function attributes (such as inlining hints) and parameter attributes (such as read-only, aliasing hints) can also

2. 广播：两个操作数的内容会被复制足够多次，直到它们的形状完全相同；如果无法做到这一点，则会发出错误。

Listing 4. Broadcasting semantics in practice

```
int a[16], b[32, 16], c[16, 1];
// a is first reshaped to [1, 16]
// and then broadcast to [32, 16]
int x_1[32, 16] = a[newaxis, :] + b;
// Same as above but implicitly
int x_2[32, 16] = a + b;
// a is first reshaped to [1, 16]
// a is broadcast to [16, 16]
// c is broadcast to [16, 16]
int y[16, 16] = a + c;
```

3.3 编程模型

GPU上CUDA[33]代码的执行基于SPMD[4]编程模型，其中每个内核都与所谓的启动网格中一个可识别的线程块相关联。Triton编程模型与此类似，但每个内核都是单线程的——尽管会自动并行化——并且与一组因实例而异（参见图3）的全局范围相关联。这种方法使得内核更简单，其中类似CUDA的并发原语（共享内存同步、线程间通信等）不复存在。

与内核相关的全局范围可以使用内置函数 `get_global_range(axis)` 进行查询，以便创建例如指针的瓦片，如清单1所示。

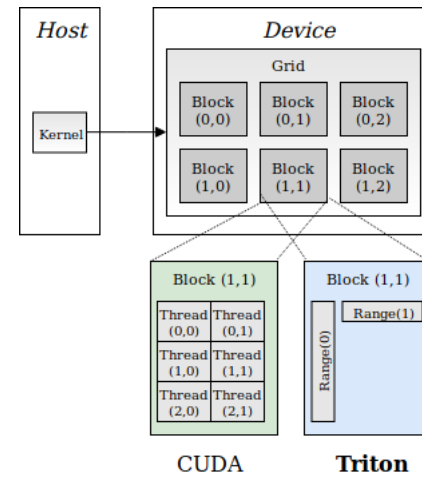


图3. CUDA与Triton编程模型的差异

4 Triton IR

Triton-IR是一种基于LLVM的中间表示（IR），其目的是提供一个适合图块级程序分析、转换和优化的环境。在这项工作中，Triton-IR程序被直接构建

清单5. Triton-IR中的 $= \max(A, 0)$ 。请注意，此处的瓦片形状是非参数化的。在本文中，它们的值由Triton-JIT实例化。

```
define kernel void @relu(float* %A, i32 %M, i32 %N) {
prologue:
  %rm = call i32<8> @get_global_range(0);
  %rn = call i32<8> @get_global_range(1);
  ; broadcast shapes
  %1 = reshape i32<8, 8> %M;
  %M0 = broadcast i32<8, 8> %1;
  %2 = reshape i32<8, 8> %N;
  %N0 = broadcast i32<8, 8> %2;
  ; broadcast global ranges
  %3 = reshape i32<8, 1> %rm;
  %rm_bc = broadcast i32<8, 8> %3;
  %4 = reshape i32<1, 8> %rn;
  %rn_bc = broadcast i32<8, 8> %4;
  ; compute mask
  %pm = icmp slt %rm_bc, %M0;
  %pn = icmp slt %rn_bc, %N0;
  %msk = and %pm, %pn;
  ; compute pointer
  %A0 = splat float*<8, 8> %A;
  %5 = getelementptr %A0, %rm_bc;
  %6 = mul %rn_bc, %M0;
  %pa = getelementptr %5, %6;
  ; compute result
  %a = load %pa;
  %_0 = splat float<8, 8> 0;
  %result = max %float %a, %_0;
  ; write back
  store fp32<8, 8> %pa, %result
}
```

尽管未来它们也可能直接从更高级别的DSL生成，但在解析过程中，它们来自Triton-C。

Triton-IR与LLVM-IR程序共享相同的高级结构（第4.1节已回顾），但前者还包含一系列扩展，这些扩展对于瓦片级数据流（第4.2节）和控制流（第4.3节）分析至关重要。这些新颖的扩展对于实施第5节概述的优化以及如第6节所示安全访问任意形状的张量至关重要。

4.1 结构

4.1.1 模块

在最高层级，Triton-IR程序由一个或多个称为模块的编译基本单元组成。这些模块彼此独立编译，最终由链接器聚合，其作用是解析前向声明并适当合并全局定义。

每个模块本身由函数、全局变量、常量以及其他杂项符号（例如元数据、函数属性）组成。

4.1.2 函数

Triton-IR函数定义包含返回类型、名称以及可能为空的参数列表。根据需要，可添加额外的可见性、对齐方式和链接说明符。函数属性（如内联提示）和参数属性（如只读、别名提示）也可被指定。

be specified, allowing compiler backends to perform more aggressive optimizations by, for instance, making better use of read-only memory caches.

This header is followed by a body composed of a list of basic blocks whose interdependencies form the *Control Flow Graph* (CFG) of the function.

4.1.3 Basic Blocks

Basic blocks are, by definition, straight-line code sequences that may only contain so-called *terminator* instructions (i.e., branching, return) at their end.

Triton-IR uses the Static Single Assignment (SSA) form, meaning that each variable in each basic block must be (1) assigned to only once and (2) defined before being used. In so doing, each basic block implicitly defines a *Data-Flow Graph* (DFG) whose different paths correspond to *use-def* chains in the program’s SSA representation. This form can be created directly from Abstract Syntax Trees (ASTs) as shown in [7].

4.2 Support for Tile-Level Data-Flow Analysis

4.2.1 Types

Multi-dimensional tiles are at the center of data-flow analysis in Triton-IR and can be declared using syntax similar to vector declarations in LLVM-IR. For example, `i32<8, 8>` is the type corresponding to 8×8 32-bit integer tiles. Note that there is no `tunable` keyword in Triton-IR, hence parametric shape values must be resolved before programs are generated. In our case, this is done by Triton-JIT’s auto-tuner (Section 5.3).

4.2.2 Instructions

Triton-IR introduces a set of *retiling* instructions whose purpose is to support broadcasting semantics as described in Section 3.2.2:

- The `reshape` instruction creates a tile of the specified shapes using data from its input argument. This is particularly useful to re-interpret variables as higher-dimensional arrays by padding their input shapes with ones in preparation of implicit or explicit broadcasting.
- The `broadcast` instruction creates a tile of the specified shapes by replicating its input argument as many times as necessary along dimensions of size 1 – as shown in Figure 4.

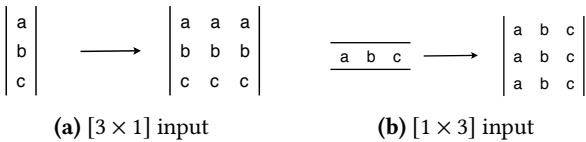


Figure 4. The `broadcast <3,3>` instruction

Usual scalar instructions (`cmp`, `getelementptr`, `add`, `load`...) were preserved and extended to signify element-wise operations on tile operands. Finally, Triton-IR also exposes specialized

arithmetic instructions for transpositions (`trans`) and matrix multiplications (`dot`).

4.3 Support for Tile-Level Control-Flow Analysis

One problem that arises from the existence of tile-level operations in Triton-IR is the inexpressibility of divergent control flow within tiles. For example, a program may need to partially guard tile-level loads against memory access violations, but this cannot be achieved using branching since tile elements cannot be accessed individually.

Listing 6. Tile-Level Predication in Triton-IR

```
;pt[i,j], pf[i,j] = (true, false) if x[i,j] < 5
;pt[i,j], pf[i,j] = (false, true) if x[i,j] >= 5
%pt, %pf = icmp slt %x, 5
@%pt %x1 = add %y, 1
@%pf %x2 = sub %y, 1
; merge values from different predicates
%x = psi i32<8,8> [%pt, %x1], [%pf, %x2]
%z = mul i32<8,8> %x, 2
```

We propose to solve this issue through the use of the Predicated SSA (PSSA) form [8] and ψ -functions [39]. This requires the addition of two instruction classes (see Listing 6) to Triton-IR:

- The `cmpp` instructions [8] are similar to usual comparison (`cmp`) instructions, except for the fact that they return two opposite predicates instead of one.
- The `psi` instruction merges instructions from different streams of predicated instructions.

5 The Triton-JIT compiler

The goal of Triton-JIT is to simplify and compile Triton-IR programs into efficient machine code, via a set of machine-independent (Section 5.1) and machine-dependent (Section 5.2) passes backed by an auto-tuning engine (Section 5.3).

5.1 Machine-Independent Passes

5.1.1 Pre-Fetching

Tile-level memory operation inside loops can be problematic, as they may induce severe latency that cannot be hidden in the absence of enough independent instructions. It is however possible to mitigate this problem in Triton-IR directly by detecting loops and adding adequate prefetching code where necessary (See Listing 7).

Listing 7. Automatic pre-fetching

```
B0:
%p0 = getelementptr %1, %2
%x0 = load %p0
B1:
%p = phi [%p0,B0], [%p1,B1]
%x = phi [%x0,B0], [%x1,B1]
%x = load %p
; increment pointer
%p1 = getelementptr %p, %3
; prefetching
%x1 = load %p
```

可以指定，允许编译器后端执行更激进的优化，例如，通过更好地利用只读内存缓存。

该头部之后是一个由基本块列表构成的主体，这些基本块之间的相互依赖关系形成了函数的控制流图（CFG）。

4.1.3 基本块

根据定义，基本块是直线代码序列，其末尾只能包含所谓的终结指令（即分支、返回）。

Triton-IR采用静态单赋值（SSA）形式，这意味着每个基本块中的每个变量必须（1）仅被赋值一次，且（2）在使用前已定义。通过这种方式，每个基本块隐式定义了一个数据流图（DFG），其不同路径对应程序中SSA表示的使用-定义链。如文献[7]所示，这种形式可以直接从抽象语法树（AST）生成。

4.2 对瓦片级数据流分析的支持

4.2.1 类型

多维瓦片是Triton-IR中数据流分析的核心，其声明语法类似于LLVM-IR中的向量声明。例如，`i32<8, 8>`类型对应 8×8 32位整数瓦片。请注意，Triton-IR中没有 `tunable` 关键字，因此必须在程序生成前解析参数化形状值。在我们的实现中，这项工作由Triton-JIT的自动调优器完成（见第5.3节）。

4.2.2 指令

Triton-IR引入了一组重铺指令，其目的是支持第3.2.2节所述的广播语义：

- `reshape`指令使用其输入参数的数据创建一个指定形状的瓦片。这在将变量重新解释为高维数组时特别有用，通过在其输入形状中填充1，为隐式或显式广播做准备。
- `broadcast`指令通过沿尺寸为1的维度根据需要进行多次复制其输入参数来创建指定形状的图块——如图4所示。

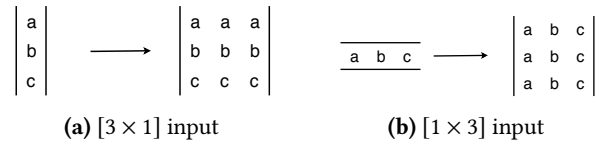


图4. `broadcast <3,3>`指令

通常的标量指令（`cmp`、`getelementptr`、`add`、`load`...）被保留并扩展，用于表示对瓦片操作数的逐元素运算。最后，Triton-IR还公开了专门的

用于转置（`trans`）和矩阵乘法（`dot`）的算术指令。

4.3 对瓦片级控制流分析的支持

Triton-IR中存在瓦片级操作带来的一个问题是，瓦片内部无法表达发散控制流。例如，程序可能需要部分保护瓦片级加载以避免内存访问违规，但由于瓦片元素无法单独访问，这无法通过分支实现。

Listing 6. Tile-Level Predication in Triton-IR

```
;pt[i,j], pf[i,j] = (true, false) if x[i,j] < 5
;pt[i,j], pf[i,j] = (false, true) if x[i,j] >= 5
%pt, %pf = icmp slt %x, 5
@%pt %x1 = add %y, 1
@%pf %x2 = sub %y, 1
; merge values from different predicates
%x = psi i32<8,8> [%pt, %x1], [%pf, %x2]
%z = mul i32<8,8> %x, 2
```

我们提出通过使用谓词SSA（PSSA）形式[8]和 ψ 函数[39]来解决这个问题。这需要在Triton-IR中添加两个指令类别（见列表6）：

- `cmpp`指令[8]与常规比较(`cmp`)指令类似，不同之处在于它们返回两个相反的谓词而非一个。
- `psi`指令合并了来自不同谓词指令流中的指令。

5 Triton-JIT编译器

Triton-JIT的目标是通过一组机器无关（第5.1节）和机器相关（第5.2节）的优化流程，结合自动调优引擎（第5.3节）的支持，将Triton-IR程序简化为高效机器码。

5.1 与机器无关的遍

5.1.1 预取

在循环内部进行瓦片级内存操作可能会带来问题，因为它们可能引发严重的延迟，在没有足够独立指令的情况下无法隐藏。然而，通过直接在Triton-IR中检测循环并在必要时添加适当的预取代码，可以缓解这个问题（参见清单7）。

清单7. 自动预取

```
B0:
%p0 = getelementptr %1, %2
%x0 = load %p0
B1:
%p = phi [%p0,B0], [%p1,B1]
%x = phi [%x0,B0], [%x1,B1]
%x = load %p
; increment pointer
%p1 = getelementptr %p, %3
; prefetching
%x1 = load %p
```

5.1.2 Tile-Level Peephole Optimization

The presence of tile-level operations in Triton-IR offers new opportunities for peephole [29] optimizers. For instance, chains of transpositions can be simplified using the identity $X = (X^T)^T$ for any tile X . We believe that other algebraic properties related to e.g., diagonal tiles could also be exploited in the future.

5.2 Machine-Dependent Passes

We now present a set of optimization passes for machines that follow the high-level model shown in Figure 5. Specifically, the optimizations performed by Triton-JIT consist of (1) hierarchical tiling, (2) memory coalescing, (3) shared memory allocation and (4) shared memory synchronization.

5.2.1 Hierarchical Tiling

Nested tiling strategies (see Figure 5) aim at decomposing tiles into micro-tiles and eventually nano-tiles in order to fit a machine's compute capabilities and memory hierarchy as tightly as possible. While this technique is routinely used in auto-tuning frameworks [34, 40], the structure of Triton-IR makes it possible to automatically enumerate and optimize valid nested tiling configurations for any expressible program (and without the need for polyhedral machinery).

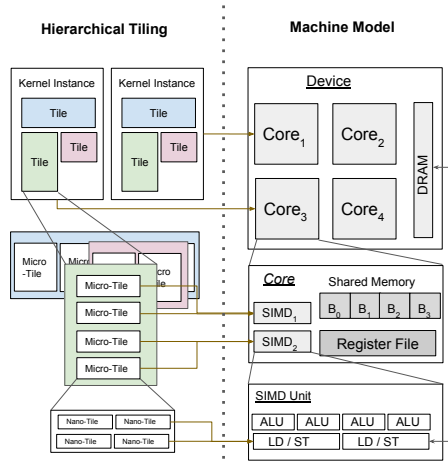


Figure 5. Hierarchical Tiling in the Triton-IR Machine Model

5.2.2 Memory Coalescing

Memory accesses are said to be *coalesced* when adjacent threads simultaneously access nearby memory locations. This is important because memory is usually retrieved in large blocks from DRAM.

Because Triton-IR programs are single-threaded and automatically parallelized, our compiler backend is able to order threads internally within each micro-tile so as to avoid uncoalesced memory accesses when possible. This strategy reduces the number of memory transactions necessary to load a tile column (see Figure 6).

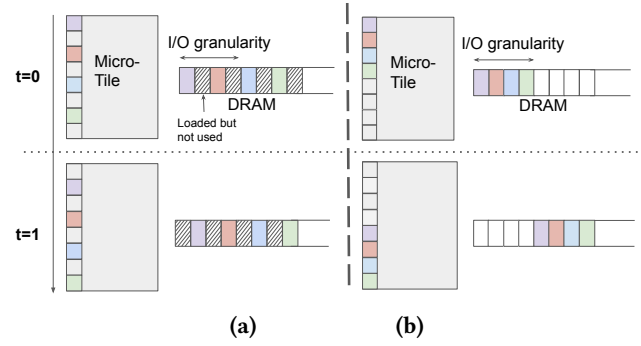


Figure 6. Uncoalesced (a) and coalesced (b) DRAM accesses. Different threads are shown in different colors.

5.2.3 Shared Memory Allocation

Tile-level operations that have high arithmetic intensity (e.g., *dot*) can benefit from temporarily storing their operands in fast shared memory. The purpose of the Shared Memory Allocation pass is to determine when and where a tile should be stashed to this space. This can be done, as illustrated in Figure 7, by first calculating the *live range* of each variable of interest, and then using the linear-time storage allocation algorithm proposed in [15].

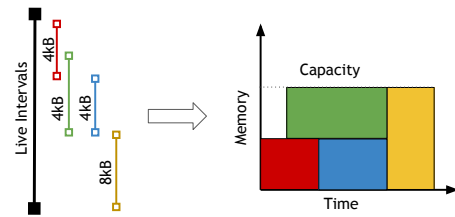


Figure 7. Shared Memory Allocation

5.2.4 Shared Memory Synchronization

Reads from and write to shared memory are asynchronous in our machine model. The goal of the Shared Memory Synchronization pass automatically inserts *barriers* in the generated GPU source code so as to preserve program correctness. This is done by detecting read-after-writes (RAW) and write-after-read (WAR) hazards using forward data-flow analysis with the following data-flow equations:

$$\begin{aligned} in_s^{(RAW)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(RAW)} \\ in_s^{(WAR)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(WAR)} \\ out_s^{(RAW)} &= \begin{cases} \emptyset & \text{if } in_s^{(RAW)} \cap read(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(RAW)} \cup write(s) & \text{otherwise} \end{cases} \\ out_s^{(WAR)} &= \begin{cases} \emptyset & \text{if } in_s^{(WAR)} \cap write(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(WAR)} \cup read(s) & \text{otherwise} \end{cases} \end{aligned}$$

5.1.2 瓦片级窥孔优化

Triton-IR中瓦片级操作的存在为窥孔[29]优化器提供了新的机遇。例如，可以利用恒等式 $X = (X^T)^T$ 简化转置链，其中 X 为任意瓦片。我们相信未来还可利用其他代数性质，例如与对角瓦片相关的特性。

5.2 机器相关处理阶段

我们现在为遵循图5所示高层模型的机器提出一系列优化步骤。具体来说，Triton-JIT执行的优化包括：(1) 分层平铺，(2) 内存合并，(3) 共享内存分配，以及(4) 共享内存同步。

5.2.1 层次化平铺

嵌套平铺策略（见图5）旨在将瓦片分解为微瓦片乃至纳米瓦片，以尽可能紧密地适配机器的计算能力和内存层次结构。虽然该技术在自动调优框架中已被常规使用[34, 40]，但Triton-IR的结构使得能够自动枚举并优化任何可表达程序的有效嵌套平铺配置（且无需依赖多面体计算工具）。

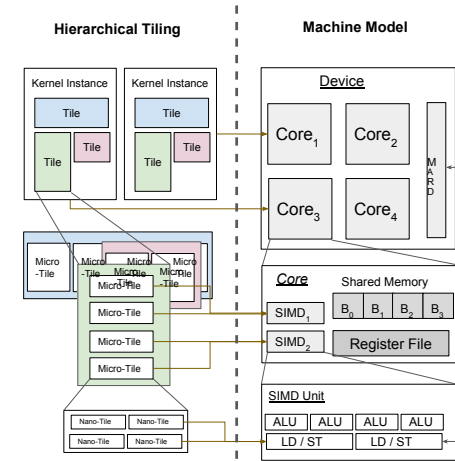


图5. Triton-IR机器模型中的层次化平铺

5.2.2 内存合并

当相邻线程同时访问相近的内存位置时，这些内存访问被称为合并访问。这一点很重要，因为内存通常是以大块的形式从DRAM中获取的。

由于Triton-IR程序是单线程且自动并行化的，我们的编译器后端能够在每个微瓦片内部对线程进行排序，从而尽可能避免非合并的内存访问。这一策略减少了加载瓦片列所需的内存事务数量（见图6）。

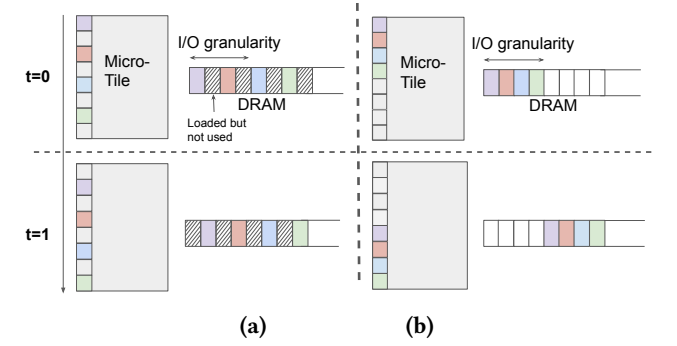


图6. 未合并 (a) 与已合并 (b) 的DRAM访问。不同线程以不同颜色显示。

5.2.3 共享内存分配

具有高算术强度的瓦片级操作（例如，*dot*）可以通过将其操作数临时存储在快速的共享内存中而受益。共享内存分配通道的目的是确定何时以及将瓦片存储到此空间。如图7所示，这可以通过首先计算每个关注变量的存活范围，然后使用[15]中提出的线性时间存储分配算法来实现。

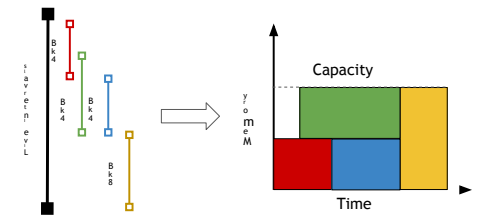


图7. 共享内存分配

5.2.4 共享内存同步

在我们的机器模型中，对共享内存的读写是异步的。共享内存同步通道的目标是自动在生成的GPU源代码中插入屏障，以保持程序的正确性。这是通过使用前向数据流分析检测读后写（RAW）和写后读（WAR）风险来实现的，其数据流方程如下：

$$\begin{aligned} in_s^{(RAW)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(RAW)} \\ in_s^{(WAR)} &= \bigcup_{p \in \text{pred}(s)} out_p^{(WAR)} \\ out_s^{(RAW)} &= \begin{cases} \emptyset & \text{if } in_s^{(RAW)} \cap read(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(RAW)} \cup write(s) & \text{otherwise} \end{cases} \\ out_s^{(WAR)} &= \begin{cases} \emptyset & \text{if } in_s^{(WAR)} \cap write(s) \neq \emptyset \text{ (barrier)} \\ in_s^{(WAR)} \cup read(s) & \text{otherwise} \end{cases} \end{aligned}$$

5.3 Auto-tuner

Traditional auto-tuners [42, 45] typically rely on hand-written parameterized code templates to achieve good performance on pre-defined workloads. By contrast, Triton-JIT can extract optimization spaces directly from Triton-IR programs by simply concatenating meta-parameters associated with each of the above optimization passes.

In this work, only the Hierarchical Tiling pass is considered, leading to no more than 3 tiling parameters per dimension per tile. These parameters are then optimized using an exhaustive search over powers of two between (a) 32 and 128 for tile sizes; (b) 8 and 32 for micro-tile sizes; and (c) 1 and 4 for nano-tile sizes. Better auto-tuning methods could be used in the future.

6 Numerical Experiments

In this section we evaluate the performance Triton on various workloads from the Deep Learning literature. We used an NVIDIA GeForce GTX1070 and compared our system against the most recent vendor libraries (cuBLAS 10.0, cuDNN 7.0) as well as related compiler technology (Auto-TVM, TC, PlaidML). When applicable, we auto-tuned these DSLs for each individual problem size following official documentation guidelines.

6.1 Matrix Multiplication

Matrix multiplication tasks of the form: $A = D \times W^T$ ($D \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{N \times K}$) are at the heart of neural network computations. Here we consider a variety of tasks from recurrent (DeepSpeech2 [3]) and transformer [44] neural networks; we report their performance in Figure 8.

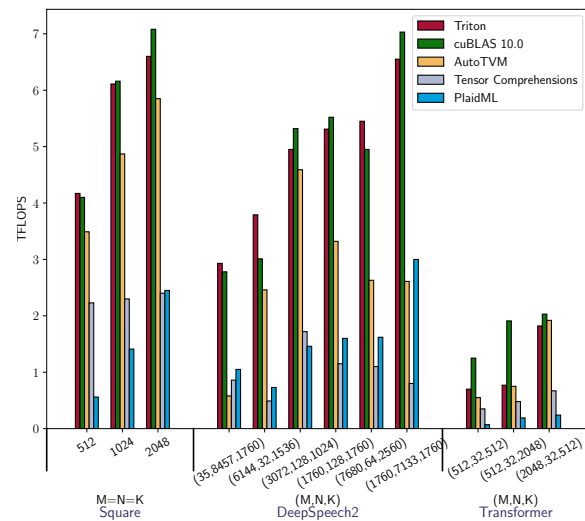


Figure 8. Performance of matrix multiplication

Triton and cuBLAS are generally on par with each other, and achieve more than 90% of the device's peak performance

on certain tasks. CuBLAS, however, remains faster than Triton on shallow transformer neural networks thanks to the use of a 3D algorithm [2] which splits deep reductions into independent chunks to provide more parallelism when M and N are too small. Otherwise, existing DSLs are 2-3x slower than our solution – except for TVM (< 2x slower) when input shapes are multiples of 32.

6.2 Convolutions

Convolutional Neural Networks (CNNs) are an important class of machine learning models which should be well supported by DSLs and compilers. They are based around convolutional layers (Figure 9a) whose implementation as matrix multiplication (Figure 9b) is necessary to make use of specialized tensor-processing hardware – yet unsupported by existing DSLs. Here we benchmark a Triton re-implementation of cuDNN's "IMPLICIT_GEMM" algorithm (Section 6.2.1) and provide the first fused kernel available for shifted convolutions (Section 6.2.2). We implemented these routines using look-up tables of pointer increments, as shown in Listing 8.

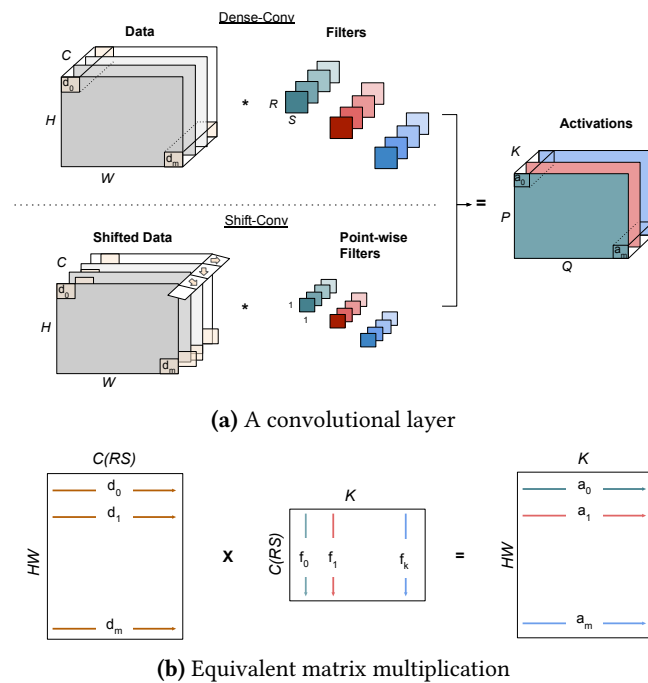


Figure 9. Dense and shifted convolutional layers (a) viewed as matrix multiplication (b)

6.2.1 Dense Convolutions

The convolutional layers considered in this subsection are from the Deep Learning literature and shown in Table 1.

As shown in Figure 10, Triton outperforms cuDNN's implementation of IMPLICIT_GEMM for ResNet. This may be due to the fact that cuDNN also maintains better algorithms for 3×3 convolutions (i.e., Winograd [25]), thereby leaving

5.3 自动调谐器

传统的自动调优器[42, 45]通常依赖手写的参数化代码模板，以在预定义的工作负载上实现良好性能。相比之下，Triton-JIT能够通过简单拼接与上述各优化通道相关联的元参数，直接从Triton-IR程序中提取优化空间。

在这项工作中，仅考虑了分层平铺过程，导致每个维度每个瓦片最多只有3个平铺参数。这些参数随后通过穷举搜索在以下范围内进行优化：(a) 瓦片尺寸在2的幂次方中取32至128；(b) 微瓦片尺寸在2的幂次方中取8至32；(c) 纳米瓦片尺寸在2的幂次方中取1至4。未来可采用更优的自动调优方法。

6 数值实验

在本节中，我们评估了Triton在深度学习文献中各种工作负载上的性能。我们使用了NVIDIA GeForce GTX1070显卡，并将我们的系统与最新的厂商库（cuBLAS 10.0、cuDNN 7.0）以及相关编译器技术（Auto-TVM、TC、PlaidML）进行了比较。在适用的情况下，我们根据官方文档指南，针对每个具体问题规模对这些领域特定语言进行了自动调优。

6.1 矩阵乘法

形式为 $A = D \times W^T$ ($D \in \mathbb{R}^{M \times K}$, $W \in \mathbb{R}^{N \times K}$) 的矩阵乘法任务是神经网络计算的核心。在此，我们考虑了来自循环网络（DeepSpeech2 [3])和Transformer [44]神经网络的多种任务；我们在图8中报告了它们的性能表现。

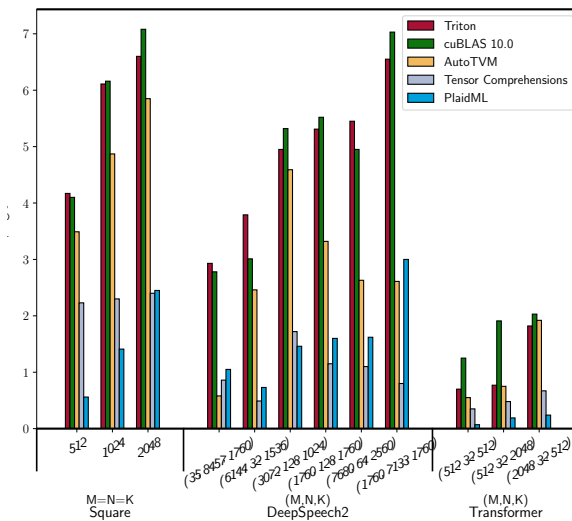


图8. 矩阵乘法性能

Triton与cuBLAS通常表现相当，都能达到设备峰值性能的90%以上。

在某些任务上。然而，由于使用了3D算法[2]，CuBLAS在浅层Transformer神经网络上仍然比Triton更快，该算法将深度归约分割成独立的块，以便在M和N太小时提供更多的并行性。否则，现有的DSL比我们的解决方案慢2-3倍——除了TVM（<慢2倍），当输入形状是32的倍数时。

6.2 卷积

卷积神经网络（CNNs）是一类重要的机器学习模型，应得到领域特定语言（DSLs）和编译器的良好支持。其核心是卷积层（图9a），将其实现为矩阵乘法（图9b）对于利用专用张量处理硬件至关重要——然而现有DSLs尚未支持此功能。在此，我们对cuDNN的"IMPLICIT_GEMM"算法（第6.2.1节）的Triton重实现进行基准测试，并首次为移位卷积提供了融合内核（第6.2.2节）。我们通过指针增量查找表实现了这些例程，如代码清单8所示。

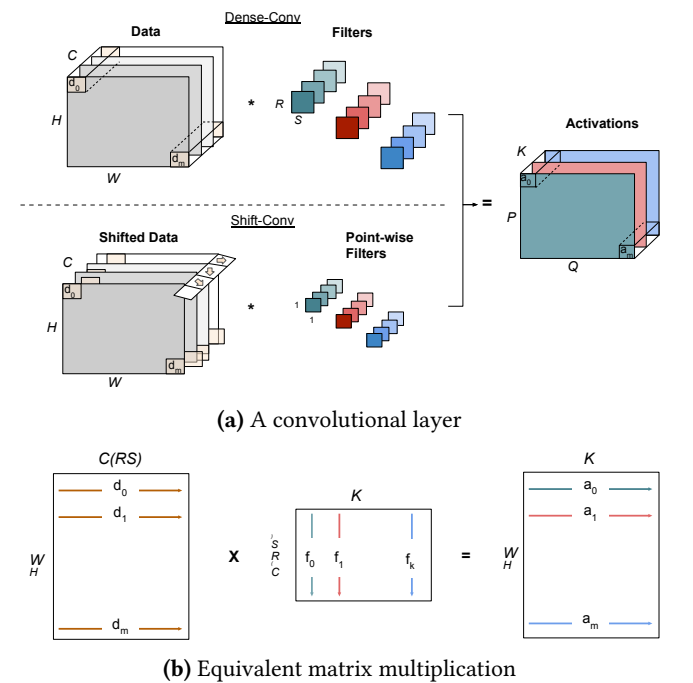


图9. 密集与平移卷积层 (a) 视为矩阵乘法 (b)

6.2.1 密集卷积

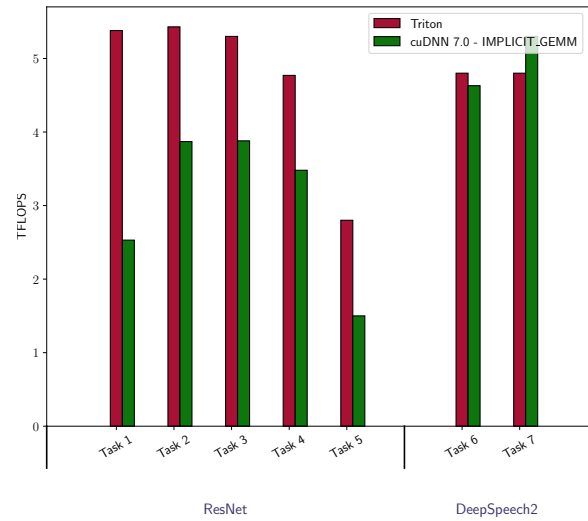
本小节考虑的卷积层来自深度学习文献，如表1所示。

如图10所示，Triton在ResNet上的表现优于cuDNN的IMPLICIT_GEMM实现。这可能是因为cuDNN还为 3×3 卷积（即Winograd[25]）保留了更好的算法，从而使得

	H	W	C	B	K	R	S	Application
Task 1	112	112	64	4	128	3	3	ResNet [19]
Task 2	56	56	128	4	256	3	3	ResNet
Task 3	28	28	256	4	512	3	3	ResNet
Task 4	14	14	512	4	512	3	3	ResNet
Task 5	7	7	512	4	512	3	3	ResNet
Task 6	161	700	1	8	64	5	5	DeepSpeech2
Task 7	79	341	32	8	32	5	10	DeepSpeech2

Table 1. Convolution tasks considered in this paper

little engineering resources for optimizing kernels of lesser importance. When fast algorithms are not available (e.g., DeepSpeech2), cuDNN and Triton are on par.

**Figure 10.** Performance of implicit matrix multiplication

6.2.2 Shift Convolutions

We finally consider an implementation of Task1-5 from Table 1 as shifted convolutions – a novel approach to CNNs (see Figure 9a). We compare our implementation of a fused shift-conv module in Triton (Listing 8) against that of a naive implementation relying on a hand-written shift kernel and a separate call to cuBLAS. We also report the maximum attainable performance when shift is not done (i.e., 1×1 convolution). As we can see in Figure 11, our Triton implementation is able to almost entirely hide the cost of shifting.

7 Conclusions

In this paper we presented Triton, an open-source language and compiler for expressing and compiling tiled neural network computations into efficient machine code. We showed that the addition of just a few data- and control-flow extensions to LLVM-IR could enable various tile-level optimization passes which jointly lead to performance on-par with vendor libraries. We also proposed Triton-C, a higher-level language in which we were able to concisely implement efficient kernels for novel neural network architectures for CNNs.

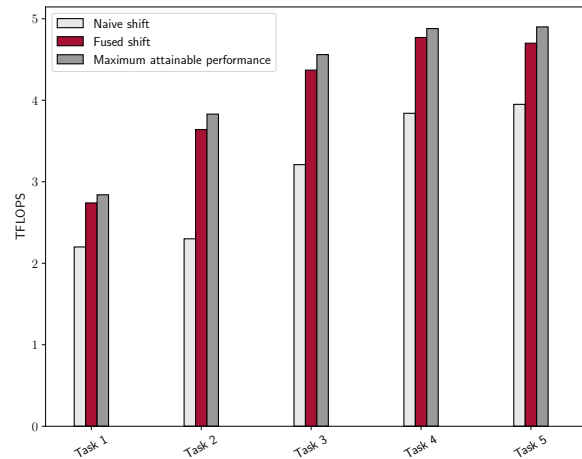
Listing 8. shift-convolutions in Triton-C

```
const tunable int TM = {16, 32, 64, 128};
const tunable int TN = {16, 32, 64, 128};
const tunable int TK = {8};

__constant__ int* delta = alloc_const int[512];

for(int c = 0; c < C; c++)
    delta[c] = c*H*W + shift_h[c]*W + shift_w[c]

void shift_conv(restrict read_only float *a,
               restrict read_only float *b, float *c,
               int M, int N, int K){
    int rxa[TM] = get_global_range[TM](0);
    int ryb[TN] = get_global_range[TN](1);
    int rka[TK] = 0 ... TK;
    int rkb[TK] = 0 ... TK;
    float C[TM, TN] = 0;
    float* pxa[TM, TK] = a + rxa[:, newaxis];
    float* pb[TN, TK] = b + ryb[:, newaxis] + rkb*N;
    __constant__ int* pd[TK] = delta + rka;
    for(int k = K; k > 0; k = k - TK){
        int delta[TK] = *pd;
        float *pa[TM, TK] = pxa + delta[newaxis, :];
        float a[TM, TK] = *pa;
        float b[TN, TK] = *pb;
        C = dot(a, trans(b), C);
        pb = pb + TK*N;
        pd = pd + TK;
    }
    int rxc[TM] = get_global_range[TM](0);
    int ryc[TN] = get_global_range[TN](1);
    float* pc[TM, TN] = c + rxc[:, newaxis] + ryc*M;
    bool checkc0[TM] = rxc < M;
    bool checkc1[TN] = ryc < N;
    bool checkc[TM, TN] = checkc0[:, newaxis] && checkc1;
    @checkc *pc = C;
}
```

**Figure 11.** Performance of shifted convolutions in Triton

Directions of future work includes support for tensor cores, implementation of quantized kernels [26] and integration into higher level DSLs.

8 Acknowledgements

This work was supported by the NVIDIA Graduate Fellowship.

	H	W	C	B	K	R	S	Application
Task 1	112	112	64	4	128	3	3	ResNet [19]
Task 2	56	56	128	4	256	3	3	ResNet
Task 3	28	28	256	4	512	3	3	ResNet
Task 4	14	14	512	4	512	3	3	ResNet
Task 5	7	7	512	4	512	3	3	ResNet
Task 6	161	700	1	8	64	5	5	DeepSpeech2
Task 7	79	341	32	8	32	5	10	DeepSpeech2

表1. 本文考虑的卷积任务

用于优化重要性较低的内核的工程资源很少。当快速算法不可用时（例如DeepSpeech2），cuDNN和Triton表现相当。

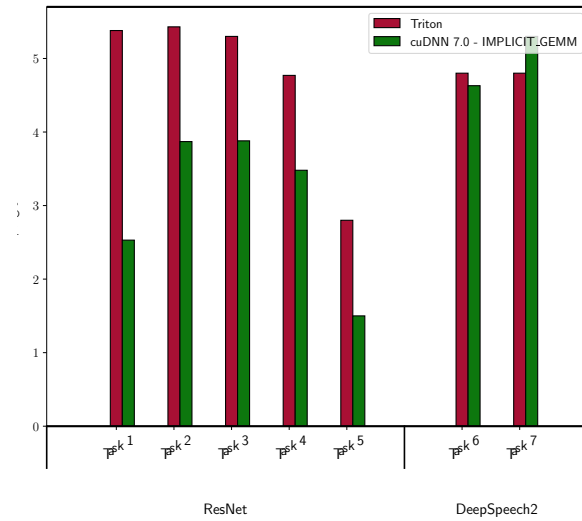


图10. 隐式矩阵乘法的性能

6.2.2 移位卷积

我们最终将表1中的Task1-5实现为移位卷积——这是一种新颖的CNN方法（见图9a）。我们将Triton中融合移位卷积模块的实现（代码清单8）与依赖手写移位内核并单独调用cuBLAS的朴素实现进行了比较。同时报告了未执行移位操作时（即 1×1 卷积）可达到的最大性能。如图11所示，我们的Triton实现几乎能完全隐藏移位操作的开销。

7 结论

本文介绍了Triton，一种用于表达并将分块神经网络计算编译为高效机器码的开源语言与编译器。我们证明了在LLVM-IR中仅添加少量数据流与控制流扩展，即可实现多种分块级优化过程，这些优化共同使性能达到与厂商库相当的水平。我们还提出了Triton-C，一种更高级的语言，通过它我们能够对CNN等新型神经网络架构简洁地实现高效内核。

Listing 8. shift-convolutions in Triton-C

```
const tunable int TM = {16, 32, 64, 128};
const tunable int TN = {16, 32, 64, 128};
const tunable int TK = {8};

__constant__ int* delta = alloc_const int[512];

for(int c = 0; c < C; c++)
    delta[c] = c*H*W + shift_h[c]*W + shift_w[c]

void shift_conv(restrict read_only float *a,
               restrict read_only float *b, float *c,
               int M, int N, int K){
    int rxa[TM] = get_global_range[TM](0);
    int ryb[TN] = get_global_range[TN](1);
    int rka[TK] = 0 ... TK;
    int rkb[TK] = 0 ... TK;
    float C[TM, TN] = 0;
    float* pxa[TM, TK] = a + rxa[:, newaxis];
    float* pb[TN, TK] = b + ryb[:, newaxis] + rkb*N;
    __constant__ int* pd[TK] = delta + rka;
    for(int k = K; k > 0; k = k - TK){
        int delta[TK] = *pd;
        float *pa[TM, TK] = pxa + delta[newaxis, :];
        float a[TM, TK] = *pa;
        float b[TN, TK] = *pb;
        C = dot(a, trans(b), C);
        pb = pb + TK*N;
        pd = pd + TK;
    }
    int rxc[TM] = get_global_range[TM](0);
    int ryc[TN] = get_global_range[TN](1);
    float* pc[TM, TN] = c + rxc[:, newaxis] + ryc*M;
    bool checkc0[TM] = rxc < M;
    bool checkc1[TN] = ryc < N;
    bool checkc[TM, TN] = checkc0[:, newaxis] && checkc1;
    @checkc *pc = C;
}
```

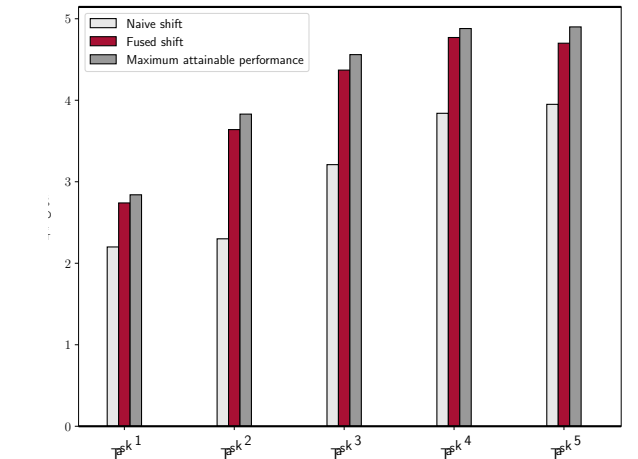


图11. Triton中移位卷积的性能

未来工作的方向包括支持张量核心、实现量化内核[26]以及集成到更高级别的DSL中。

8 致谢

本工作得到了NVIDIA研究生奖学金的支持。

References

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis, Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow: A System for Large-scale Machine Learning. In *Proceedings of the 12th USENIX Conference on Operating Systems Design and Implementation (OSDI’16)*.
- [2] R. C. Agarwal, S. M. Balle, F. G. Gustavson, M. Joshi, and P. Palkar. 1995. A three-dimensional approach to parallel matrix multiplication. *IBM Journal of Research and Development* 39, 5 (Sep. 1995), 575–582. <https://doi.org/10.1147/rd.395.0575>
- [3] Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski, Adam Coates, Greg Diamos, Erich Elsen, Jesse Engel, Linxi Fan, Christopher Fougner, Tony Han, Awni Y. Hannun, Billy Jun, Patrick LeGresley, Libby Lin, Sharan Narang, Andrew Y. Ng, Sherjil Ozair, Ryan Prenger, Jonathan Raiman, Sanjeev Satheesh, David Seetapun, Shubho Sengupta, Yi Wang, Zhiqian Wang, Chong Wang, Bo Xiao, Dani Yogatama, Jun Zhan, and Zhenyao Zhu. 2015. Deep Speech 2: End-to-End Speech Recognition in English and Mandarin. *CoRR* abs/1512.02595 (2015). arXiv:1512.02595 <http://arxiv.org/abs/1512.02595>
- [4] M. Auguin and F. Larbey. [n. d.]. Opsila: an advanced SIMD for numerical analysis and signal processing. ([n. d.]), 311–318.
- [5] Bin Bao and Chen Ding. 2013. Defensive Loop Tiling for Shared Cache. In *Proceedings of the 2013 IEEE/ACM International Symposium on Code Generation and Optimization (CGO) (CGO ’13)*. IEEE Computer Society, Washington, DC, USA, 1–11.
- [6] Muthu Manikandan Baskaran, Albert Hartono, Sanket Tavarageri, Thomas Henretty, J. Ramanujam, and P. Sadayappan. 2010. Parameterized Tiling Revisited. In *Proceedings of the 8th Annual IEEE/ACM International Symposium on Code Generation and Optimization (CGO ’10)*. ACM, New York, NY, USA, 200–209. <https://doi.org/10.1145/1772954.1772983>
- [7] Matthias Braun, Sebastian Buchwald, Sebastian Hack, Roland Leissa, Christoph Mallon, and Andreas Zwinkau. 2013. Simple and Efficient Construction of Static Single Assignment Form. In *Proceedings of the 22Nd International Conference on Compiler Construction (CC’13)*. 102–122.
- [8] Lori Carter, Beth Simon, Brad Calder, Larry Carter, and Ferrante. [n. d.]. Predicated Static Single Assignment. In *Proceedings of the PACT 1999 Conference on Parallel Architectures and Compilation Techniques*.
- [9] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MXNet: A Flexible and Efficient Machine Learning Library for Heterogeneous Distributed Systems. *CoRR* abs/1512.01274 (2015).
- [10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: End-to-End Optimization Stack for Deep Learning. *CoRR* abs/1802.04799 (2018).
- [11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM: End-to-End Optimization Stack for Deep Learning. *CoRR* abs/1802.04799 (2018).
- [12] Jack W. Davidson and Sanjay Jinturkar. 1994. Memory Access Coalescing: A Technique for Eliminating Redundant Memory Accesses. In *Proceedings of the ACM SIGPLAN 1994 Conference on Programming Language Design and Implementation (PLDI ’94)*. ACM, New York, NY, USA, 186–195. <https://doi.org/10.1145/178243.178259>
- [13] Greg Diamos, Shubho Sengupta, Bryan Catanzaro, Mike Chrzanowski, Adam Coates, Erich Elsen, Jesse Engel, Awni Hannun, and Sanjeev Satheesh. 2016. Persistent RNNs: Stashing Recurrent Weights On-Chip. In *Proceedings of The 33rd International Conference on Machine*

Learning.

- [14] Venmugil Elango, Norm Rubin, Mahesh Ravishankar, Hariharan Sandanagobalane, and Vinod Grover. 2018. Diesel: DSL for Linear Algebra and Neural Net Computations on GPUs. In *Proceedings of the 2Nd ACM SIGPLAN International Workshop on Machine Learning and Programming Languages (MAPL 2018)*. ACM, New York, NY, USA, 42–51. <https://doi.org/10.1145/3211346.3211354>
- [15] Jordan Gergov. 1999. Algorithms for Compile-time Memory Optimization. In *Proceedings of the Tenth Annual ACM-SIAM Symposium on Discrete Algorithms (SODA ’99)*.
- [16] Google Inc. 2017. *Tensorflow XLA*. <https://www.tensorflow.org/performance/xla/>
- [17] Scott Gray and Alex Radford ans Diederik P. Kingma. 2017. Gpu kernels for block-sparse weights. *CoRR* abs/1711.09224 (2017).
- [18] M. Griebl, C. Lengauer, and S. Wetzel. 1998. Code generation in the polytope model. In *Proceedings. 1998 International Conference on Parallel Architectures and Compilation Techniques (Cat. No.98EX192)*. 106–111. <https://doi.org/10.1109/PACT.1998.727179>
- [19] K. He, X. Zhang, S. Ren, and J. Sun. 2016. Deep Residual Learning for Image Recognition. In *2016 IEEE Conference on Computer Vision and Pattern Recognition (CVPR)*. 770–778. <https://doi.org/10.1109/CVPR.2016.90>
- [20] Andrew G. Howard, Menglong Zhu, Bo Chen, Dmitry Kalenichenko, Weijun Wang, Tobias Weyand, Marco Andreetto, and Hartwig Adam. 2017. MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications. *CoRR* abs/1704.04861 (2017). arXiv:1704.04861 <http://arxiv.org/abs/1704.04861>
- [21] Jianyu Huang and Robert A. van de Geijn. 2016. *BLISlab: A Sandbox for Optimizing GEMM*. FLAME Working Note #80, TR-16-13. The University of Texas at Austin, Department of Computer Science. <http://arxiv.org/pdf/1609.00076v1.pdf>
- [22] Intel AI. 2018. *PlaidML*. <https://www.intel.ai/reintroducing-plaidml>
- [23] Andrew Kerr, Duane Merrill, Julien Demouth, and John Tran. [n. d.]. CUTLASS: Fast Linear Algebra in CUDA C++. ([n. d.]). <https://devblogs.nvidia.com/cutlass-linear-algebra-cuda/>
- [24] Alex Krizhevsky, Ilya Sutskever, and Geoffrey E. Hinton. 2012. ImageNet Classification with Deep Convolutional Neural Networks. In *Proceedings of the 25th International Conference on Neural Information Processing Systems - Volume 1 (NIPS’12)*.
- [25] Andrew Lavin and Scott Gray. 2016. Fast Algorithms for Convolutional Neural Networks. (CVPR’16).
- [26] Darryl D. Lin, Sachin S. Talathi, and V. Sreekanth Annapureddy. 2016. Fixed Point Quantization of Deep Convolutional Networks. In *Proceedings of the 33rd International Conference on International Conference on Machine Learning - Volume 48 (ICML’16)*. JMLR.org, 2849–2858. <http://dl.acm.org/citation.cfm?id=3045390.3045690>
- [27] Matt Martineau, Patrick Atkinson, and Simon McIntosh-Smith. 2017. *Benchmarking the NVIDIA V100 GPU and Tensor Cores*. Springer.
- [28] Bradley McDanel, Surat Teerapittayanon, and H.T. Kung. 2017. Embedded Binarized Neural Networks. In *Proceedings of the 2017 International Conference on Embedded Wireless Systems and Networks (EWSN ’17)*. Junction Publishing, USA, 168–173. <http://dl.acm.org/citation.cfm?id=3108009.3108031>
- [29] W. M. McKeeman. 1965. Peephole Optimization. *Commun. ACM* 8, 7 (July 1965), 443–444. <https://doi.org/10.1145/364995.365000>
- [30] Sanyam Mehta, Rajat Garg, Nishad Trivedi, and Pen-Chung Yew. 2016. TurboTiling: Leveraging Prefetching to Boost Performance of Tiled Codes. In *Proceedings of the 2016 International Conference on Supercomputing (ICS ’16)*. ACM, New York, NY, USA, Article 38, 12 pages.
- [31] Sharan Narang, Eric Undersander, and Gregory F. Diamos. 2017. Block-Sparse Recurrent Neural Networks. *CoRR* abs/1711.02782 (2017).
- [32] Rajib Nath, Stanimire Tomov, and Jack Dongarra. 2011. Accelerating GPU Kernels for Dense Linear Algebra. In *Proceedings of the 9th International Conference on High Performance Computing for Computational Science (VECPAR’10)*. Springer-Verlag, Berlin, Heidelberg, 83–92.

参考文献

- [1] Martín Abadi, Paul Barham, Jianmin Chen, Zhifeng Chen, Andy Davis , Jeffrey Dean, Matthieu Devin, Sanjay Ghemawat, Geoffrey Irving, Michael Isard, Manjunath Kudlur, Josh Levenberg, Rajat Monga, Sherry Moore, Derek G. Murray, Benoit Steiner, Paul Tucker, Vijay Vasudevan, Pete Warden, Martin Wicke, Yuan Yu, and Xiaoqiang Zheng. 2016. TensorFlow：一个用于大规模机器学习的系统。见《第12届USENIX操作系统设计与实现会议论文集》（OSDI’16）。[2] R. C. Agarwal, S. M. Balle, F. G. Gustavson, M. Joshi, and P. Palkar. 1995. 一种三维并行矩阵乘法方法。《IBM研发期刊》39卷5期（1995年9月），575–582. <https://doi.org/10.1147/rd.395.0575> [3] Dario Amodei, Rishita Anubhai, Eric Battenberg, Carl Case, Jared Casper, Bryan Catanzaro, Jingdong Chen, Mike Chrzanowski , Adam Coates, Greg Diamos, Erich Elsen, Jesse Engel, Linxi Fan, Christopher Fougner, Tony Han, Awni Y. Hannun, Billy Jun, Patrick LeGresley, Libby Lin, Sharan Narang, Andrew Y. Ng, Sherjil Ozair, Ryan Prenger, Jonathan Raiman, Sanjeev Satheesh, David Seetapun, Shubho Sengupta, Yi Wang, Zhiqian Wang, Chong Wang, Bo Xiao, Dani Yogatama, Jun Zhan, and Zhenyao Zhu. 2015. Deep Speech 2：英语和普通话的端到端语音识别。《CoRR》abs/1512.02595（2015）。arXiv:1512.02595 <http://arxiv.org/abs/1512.02595> [4] M. Auguin and F. Larbey. [n. d.]. Opsila：一种用于数值分析和信号处理的高级SIMD。([n. d.]), 311–318. [5] Bin Bao and Chen Ding. 2013. 面向共享缓存的防御性循环分块。见《2013年IEEE/ACM国际代码生成与优化研讨会论文集》（CGO’13）。IEEE计算机学会，华盛顿特区，美国，1–11。[6] Muthu Manikandan Baskaran, Albert Hartono, Sanket Tavarageri, Thomas Henretty, J. Ramanujam, and P. Sadayappan. 2010. 参数化分块再探讨。见《第8届IEEE/ACM国际代码生成与优化年度研讨会论文集》（CGO’10）。ACM，纽约，纽约州，美国，200–209. <https://doi.org/10.1145/1772954.1772983> [7] Matthias Braun, Sebastian Buchwald, Sebastian Hack, Roland Leissa, Christoph Mallon, and Andreas Zwinkau. 2013. 静态单赋值值形式的简单高效构建。见《第22届国际编译器构造会议论文集》（CC’13）。102–122。[8] Lori Carter, Beth Simon, Brad Calder, Larry Carter, and Ferrante. [n. d.]. 谓词静态单赋值。见《PACT 1999并行体系结构与编译技术会议论文集》。[9] Tianqi Chen, Mu Li, Yutian Li, Min Lin, Naiyan Wang, Minjie Wang, Tianjun Xiao, Bing Xu, Chiyuan Zhang, and Zheng Zhang. 2015. MX Net：一个面向异构分布式系统的灵活高效机器学习库。《CoRR》abs/1512.01274（2015）。[10] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM：深度学习端到端优化栈。《CoRR》abs/1802.04799（2018）。[11] Tianqi Chen, Thierry Moreau, Ziheng Jiang, Haichen Shen, Eddie Q. Yan, Leyuan Wang, Yuwei Hu, Luis Ceze, Carlos Guestrin, and Arvind Krishnamurthy. 2018. TVM：深度学习端到端优化栈。《CoRR》abs/1802.04799（2018）。[12] Jack W. Davidson and Sanjay Jinturkar. 1994. 内存访问合并：一种消除冗余内存访问的技术。见《ACM SIGPLAN 1994编程语言设计与实现会议论文集》（PLDI’94）。ACM，纽约，纽约州，美国，186–195. <https://doi.org/10.1145/178243.178259> [13] Greg Diamos, Shubho Sengupta, Bryan Catanzaro, Mike Chrzanowski, Adam Coates, Erich Elsen, Jesse Engel, Awni Hannun, and Sanjeev Satheesh. 2016. 持久性RNN：片上存储循环权重。见《第33届国际机器学习会议论文集》。

学习。[14] Venmugil Elango、Norm Rubin、Mahesh Ravishankar、Hariharan Sandanagobalane 和 Vinod Grover。2018。Diesel：GPU 上线性代数与神经网络计算的 DSL。载于《第二届 ACM SIGPLAN 机器学习与编程语言国际研讨会论文集》（MAPL 2018）。ACM，美国纽约州纽约市，42–51。 <https://doi.org/10.1145/3211346.3211354> [15] Jordan Gergov。1999。编译时内存优化算法。载于《第十届 ACM-SIAM 离散算法年度研讨会论文集》（SODA ’99）。[16] Google Inc.。2017。Tensorflow XLA。 <https://www.tensorflow.org/performance/xla/> [17] Scott Gray、Alex Radford 和 Diederik P. Kingma。2017。块稀疏权重重的 GPU 内核。CoRR abs/1711.09224 (2017)。[18] M. Griebl、C. Lengauer 和 S. Wetzel。1998。多面体模型中的代码生成。载于《1998 年国际并行体系结构与编译技术会议论文集》（Cat. No.98EX192）。106–111。 <https://doi.org/10.1109/PACT.1998.727179> [19] K. He、X. Zhang、S. Ren 和 J. Sun. 2016。用于图像识别的深度残差学习。载于《2016 年 IEEE 计算机视觉与模式识别会议》（CVPR）。770–778。 <https://doi.org/10.1109/CVPR.2016.90> [20] Andrew G. Howard、Menglong Zhu、Bo Chen、Dmitry Kalenichenko、Weijun Wang、Tobias Weyand、Marco Andreetto 和 Hartwig Adam。2017。MobileNets：用于移动视觉应用的高效卷积神经网络。CoRR abs/1704.04861 (2017)。arXiv:1704.04861 <http://arxiv.org/abs/1704.04861> [21] Jianyu Huang 和 Robert A. van de Geijn。2016。BLISlab：优化 GEMM 的沙盒。FLAME 工作笔记 #80，TR-16-13。德克萨斯大学奥斯汀分校计算机科学系。 <http://arxiv.org/pdf/1609.00076v1.pdf> [22] Intel AI。2018。PlaidML。 <https://www.intel.ai/reintroducing-plaidml> [23] Andrew Kerr、Duane Merrill、Julien Demouth 和 John Tran。[n. d.]. CUTLASS：CUDA C++ 中的快速线性代数。([n. d.])。 <https://devblogs.nvidia.com/cutlass-linear-algebra-cuda/> [24] Alex Krizhevsky、Ilya Sutskever 和 Geoffrey E. Hinton。2012。使用深度卷积神经网络进行 ImageNet 分类。载于《第 25 届神经信息处理系统国际会议论文集 - 第 1 卷》（NIPS’12）。[25] Andrew Lavin 和 Scott Gray。2016。卷积神经网络的快速算法。（CVPR’16）。[26] Darryl D. Lin、Sachin S. Talathi 和 V. Sreekanth Annapureddy。2016。深度卷积网络的定点量化。载于《第 33 届国际机器学习会议论文集 - 第 48 卷》（ICML’16）。JMLR.org，2849–2858。 <http://dl.acm.org/citation.cfm?id=3045390.3045690> [27] Matt Martineau、Patrick Atkinson 和 Simon McIntosh-Smith。2017。NVIDIA V100 GPU 与 Tensor Cores 基准测试。Springer。[28] Bradley McDanel、Surat Teerapittayanon 和 H.T. Kung。2017。嵌入式二值化神经网络。载于《2017 年国际嵌入式无线系统与网络会议论文集》（EWSN ’17）。Junction Publishing，美国，168–173。 <http://dl.acm.org/citation.cfm?id=3108009.3108031> [29] W. M. McKeeman。1965。窥孔优化。Commun. ACM 8, 7 (1965年7月)，443–444。 <https://doi.org/10.1145/364995.365000> [30] Sanyam Mehta、Rajat Garg、Nishad Trivedi 和 Pen-Chung Yew。2016。TurboTiling：利用预取提升分块代码性能。载于《2016 年国际超级计算会议论文集》（IC S ’16）。ACM，美国纽约州纽约市，文章 38，12 页。[31] Sharan Narang、Eric Undersander 和 Gregory F. Diamos。2017。块稀疏循环神经网络。CoRR abs/1711.02782 (2017)。[32] Rajib Nath、Stanimire Tomov 和 Jack Dongarra。2011。加速稠密线性代数的 GPU 内核。载于《第九届高性能计算科学国际会议论文集》（VECPAR’10）。Springer-Verlag，德国柏林海德堡，83–92。

<http://dl.acm.org/citation.cfm?id=1964238.1964250>

[33] John Nickolls, Ian Buck, Michael Garland, and Kevin Skadron. 2008. Scalable Parallel Programming with CUDA. *Queue* 6, 2 (March 2008), 40–53. <https://doi.org/10.1145/1365490.1365500>

[34] Cedric Nugteren. 2017. CLBlast: A Tuned OpenCL BLAS Library. *CoRR* abs/1705.05249 (2017). arXiv:1705.05249 <http://arxiv.org/abs/1705.05249>

[35] Travis Oliphant. 2006–. NumPy: A guide to NumPy. USA: Trelgol Publishing. (2006–). <http://www.numpy.org/> [Online; accessed <today>].

[36] PyTorch. 2016. . <https://github.com/pytorch/pytorch>

[37] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand, and Saman Amarasinghe. 2013. Halide: A Language and Compiler for Optimizing Parallelism, Locality, and Recomputation in Image Processing Pipelines. In *Proceedings of the 34th ACM SIGPLAN Conference on Programming Language Design and Implementation (PLDI ’13)*.

[38] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Summer Deng, Roman Dzhabarov, James Hegeman, Roman Levenstein, Bert Maher, Nadathur Satish, Jakob Olesen, Jongsoo Park, Artem Rakhov, and Misha Smelyanskiy. 2018. Glow: Graph Lowering Compiler Techniques for Neural Networks. *CoRR* abs/1805.00907 (2018).

[39] Arthur Stoutchinin and Francois de Ferriere. 2001. Efficient Static Single Assignment Form for Predication. In *Proceedings of the 34th Annual ACM/IEEE International Symposium on Microarchitecture (MICRO 34)*. IEEE Computer Society, Washington, DC, USA, 172–181. <http://dl.acm.org/citation.cfm?id=563998.564022>

[40] Philippe Tillet and David Cox. 2017. Input-aware Auto-tuning of Compute-bound HPC Kernels. In *Proceedings of the International Conference for High Performance Computing, Networking, Storage and Analysis (SC ’17)*. ACM.

[41] Didem Unat, Tan Nguyen, Weiqun Zhang, Muhammed Nufail Farooqi, Burak Bastem, George Michelogiannakis, Ann Almgren, editor="Kunkel Julian M. Shalf, John", Pavan Balaji, and Jack Dongarra. 2016". TiDA: High-Level Programming Abstractions for Data Locality Management.

[42] Field G. Van Zee and Robert A. van de Geijn. 2015. BLIS: A Framework for Rapidly Instantiating BLAS Functionality. *ACM Trans. Math. Softw.* 41, 3 (June 2015).

[43] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, and Albert Cohen. 2018. Tensor Comprehensions: Framework-Agnostic High-Performance Machine Learning Abstractions. *CoRR* abs/1802.04730 (2018).

[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, and Illia Polosukhin. 2017. Attention Is All You Need. *CoRR* abs/1706.03762 (2017). arXiv:1706.03762 <http://arxiv.org/abs/1706.03762>

[45] R. Clint Whaley, Antoine Petitet, and Jack J. Dongarra. 2000. Automated Empirical Optimization of Software and the ATLAS Project. *PARALLEL COMPUTING* 27 (2000), 2001.

[46] Samuel Williams, Andrew Waterman, and David Patterson. 2009. Roofline: An Insightful Visual Performance Model for Multicore Architectures. *Commun. ACM* 52, 4 (April 2009), 65–76. <https://doi.org/10.1145/1498765.1498785>

[47] Bichen Wu, Alvin Wan, Xiangyu Yue, Peter H. Jin, Sicheng Zhao, Noah Golmant, Amir Gholaminejad, Joseph Gonzalez, and Kurt Keutzer. 2017. Shift: A Zero FLOP, Zero Parameter Alternative to Spatial Convolutions. *CoRR* abs/1711.08141 (2017).

[48] Jiacheng Zhao, Huimin Cui, Yalin Zhang, Jingling Xue, and Xiaobing Feng. 2018. Revisiting Loop Tiling for Datacenters: Live and Let Live. In *Proceedings of the 2018 International Conference on Supercomputing (ICS ’18)*. ACM, New York, NY, USA, 328–340.

<http://dl.acm.org/citation.cfm?id=1964238.1964250> [33] John Nickolls, Ian Buck, Michael Garland 和 Kevin Skadron. 2008. 使用 CUDA 进行可扩展并行编程。Queue 6, 2 (2008年3月), 40–53. <https://doi.org/10.1145/1365490.1365500> [34] Cedric Nugteren. 2017. CLBlast: 一个调优的 OpenCL BLAS 库。CoRR abs/1705.05249 (2017). arXiv:1705.05249 <http://arxiv.org/abs/1705.05249> [35] Travis Oliphant. 2006–。NumPy: NumPy 指南。美国: Trelgol Publishing. (2006–)。http://www.numpy.org/ [在线; 访问于 <今天>]。[36] PyTorch. 2016. <https://github.com/pytorch/pytorch> [37] Jonathan Ragan-Kelley, Connelly Barnes, Andrew Adams, Sylvain Paris, Frédo Durand 和 Saman Amarasinghe. 2013。Halide: 一种用于优化图像处理管道中并行性、局部性和重计算的语言和编译器。载于第34届 ACM SIGPLAN 编程语言设计与实现会议论文集 (PLDI ’13)。[38] Nadav Rotem, Jordan Fix, Saleem Abdulrasool, Summer Deng, Roman Dzhabarov, James Hegeman, Roman Levenstein, Bert Maher, Nadathur Satish, Jakob Olesen, Jongsoo Park, Artem Rakhov 和 Misha Smelyanskiy. 2018. Glow: 神经网络的图降低编译器技术。CoRR abs/1805.00907 (2018)。[39] Arthur Stoutchinin 和 Francois de Ferriere. 2001。用于谓词的高效静态单赋值形式。载于第34届 ACM/IEEE 微架构国际研讨会论文集 (MICRO 34)。IEEE Computer Society, 华盛顿特区, 美国, 172–181。http://dl.acm.org/citation.cfm?id=563998.564022 [40] Philippe Tillet 和 David Cox. 2017。计算密集型高性能计算内核的输入感知自动调优。载于国际高性能计算、网络、存储和分析会议论文集 (SC ’17)。ACM。

[41] Didem Unat, Tan Nguyen, Weiqun Zhang, Muhammed Nufail Farooqi, Burak Bastem, George Michelogiannakis, Ann Almgren, 编辑="Kunkel Julian M. Shalf, John", Pavan Balaji, 与 Jack Dongarra. 2016. TiDA: 面向数据局部性管理的高级编程抽象。[42] Field G. Van Zee 与 Robert A. van de Geijn. 2015. BLIS: 快速实例化BLAS功能的框架。ACM Trans. Math. Softw. 41, 3 (2015年6月)。[43] Nicolas Vasilache, Oleksandr Zinenko, Theodoros Theodoridis, Priya Goyal, Zachary DeVito, William S. Moses, Sven Verdoolaege, Andrew Adams, 与 Albert Cohen. 2018. 张量理解: 框架无关的高性能机器学习抽象。CoRR abs/1802.04730 (2018)。[44] Ashish Vaswani, Noam Shazeer, Niki Parmar, Jakob Uszkoreit, Llion Jones, Aidan N. Gomez, Lukasz Kaiser, 与 Illia Polosukhin. 2017. 注意力机制就是一切所需。CoRR abs/1706.03762 (2017). arXiv:1706.03762 <http://arxiv.org/abs/1706.03762>[45] R. Clint Whaley, Antoine Petitet, 与 Jack J. Dongarra. 2000. 软件自动化经验优化与ATLAS项目。PARALLEL COMPUTING 27 (2000), 2001。[46] Samuel Williams, Andrew Waterman, 与 David Patterson. 2009. Roofline: 一种针对多核架构的深刻可视化性能模型。Commun. ACM 52, 4 (2009年4月), 65–76. <https://doi.org/10.1145/1498765.1498785>[47] Bichen Wu, Alvin Wan, Xiangyu Yue, Peter H. Jin, Sicheng Zhao, Noah Golmant, Amir Gholaminejad, Joseph Gonzalez, 与 Kurt Keutzer. 2017. Shift: 一种零FLOP、零参数的空间卷积替代方案。CoRR abs/1711.08141 (2017)。[48] Jiacheng Zhao, Huimin Cui, Yalin Zhang, Jingling Xue, 与 Xiaobing Feng. 2018. 为数据中心重新审视循环分块: 共生共存。载于《2018年国际超级计算会议论文集》(ICS ’18)。ACM, 纽约, 纽约州, 美国, 328–340。